

Entryウィジェット validate機能 まとめ

Entryの入力検証(バリデーション)は、`register()`・`validate`・`validatecommand`の3つの要素を組み合わせて実現します。

1. 全体の仕組み(3点セット)

```
vcmd = (self.register(self.validate_number), "%P")

entry = tk.Entry(
    self,
    validate="key",
    validatecommand=vcmd
)
```

要素	役割
<code>register()</code>	Python関数を、Tcl/Tkが呼び出せる形式に変換して登録する
<code>validate</code>	検証を「いつ」実行するか、タイミングを指定する
<code>validatecommand</code>	実際に呼び出す検証関数と、渡す情報(%Pなど)を指定する

2. register() について

Tkinterでは、Entryの入力検証に使う関数はPython関数をそのまま渡すことができず、Tcl/Tkが呼び出せる形式に変換する必要があります。`register()`はその変換を行うメソッドです。

```
vcmd = (self.register(self.validate_number), "%P")
```

`self.register(self.validate_number)` は、Pythonの `self.validate_number` という関数を、Tcl側から呼び出せる特殊な識別子に変換して登録します。

1回登録すれば使い回せる: 同じ関数を何度も`register()`し直す必要はなく、一度変換した`vcmd`を複数のEntryで使い回すことができます。

```
vcmd = (self.register(self.validate_number), "%P")

entry1 = tk.Entry(self, validate="key", validatecommand=vcmd)
entry2 = tk.Entry(self, validate="key", validatecommand=vcmd)
```

3. validate: 検証タイミングの指定

`validate` だけを指定しても検証は行われません。必ず `validatecommand` とセットで設定する必要があります。

値	検証が実行されるタイミング
"none"	検証しない(デフォルト)
"focus"	フォーカスが当たった時・外れた時の両方
"focusin"	フォーカスが当たった時のみ
"focusout"	フォーカスが外れた時のみ
"key"	キー入力のたび(1文字入力・削除するたび)
"all"	上記すべてのタイミング

"key" の挙動

ユーザーが1文字入力・削除するたびに、`validatecommand`で登録した関数が呼ばれます。数字のみ入力可、といったリアルタイム制限に向いています。

重要: "key"使用時にFalseを返すと、今まさに入力しようとしていた1文字だけが取り消され、Entry全体はクリアされずその文字が入力される前の状態に戻ります。

現在の中身: "123"
ユーザーが "a" を追加入力しようとする → `validate_number("123a")` が呼ばれる
`"123a".isdigit()` は False
→ "a" の入力だけが拒否され、Entryの中身は "123" のまま

"focusout" の挙動

フォーカスが外れたタイミングでのみ検証されます。項目全体の妥当性を、入力し終わった後にまとめてチェックしたい場合に向いています。ただし`validatecommand`はTrue/Falseで入力を許可/拒否する性質のため、「不正なら入力自体を拒否する」用途になります。単に見た目の色を変えるのであれば、`bind("<FocusOut>", ...)`の方が適しています。

4. validatecommand と %記号(渡される情報)

`validatecommand`に渡す際、「検証時にどんな情報を関数に渡すか」を%から始まる特殊な記号で指定できます。

記号	内容
%P	変更が適用された後の、Entry全体の文字列(最もよく使う)

%s	変更前の、現在のEntryの文字列
%S	今回追加・削除しようとしている文字そのもの
%d	アクション種別(1=挿入, 0=削除, -1=その他)
%i	変更が起きる位置のインデックス
%V	どのタイミングで検証が呼ばれたか(key, focusin, focusoutなど)

5. 動作の流れ

- ① ユーザーがキーを押す
↓
- ② tkinterが「もしこの入力を許可したら、Entryの中身はこうなる」という文字列(%P)を計算
↓
- ③ register()で登録された検証関数に %P を渡して呼び出す
↓
- ④ 検証関数が True / False を返す
↓
- ⑤ True なら入力が反映される / False なら入力前の状態に戻る(その文字が入力されない)

6. 実装例

```
def validate_number(self, value_if_allowed):
    if value_if_allowed == "":
        return True # 空欄は許可
    try:
        int(value_if_allowed)
        return True
    except ValueError:
        return False

vcmd = (self.register(self.validate_number), "%P")

entry = tk.Entry(self, validate="key", validatecommand=vcmd)
```

例外処理を忘れずに: int(value_if_allowed)のように変換に失敗すると例外(ValueError)が発生する処理を検証関数内で使う場合、必ずtry-exceptで囲む必要があります。例外を投げってしまうと、Tkinter内部でエラーが発生し、バリデーション自体が正常に動作しなくなることがあります。

7. isdigit()を使う場合の注意点

数字判定によく使われる `isdigit()` には、以下のような制約・注意点があります。

- マイナス記号・小数点は「数字」として扱われない(`"-123".isdigit()` は False)
- 全角数字や丸数字などもTrueになる場合がある(`"①".isdigit()` は True だが `int("①")` はエラーになる)

より厳密に「int変換できるか」を確認したい場合は、try-exceptで実際に変換を試みる方が確実です。

```
def is_valid_integer(value):
    try:
        int(value)
        return True
    except ValueError:
        return False
```

半角数字のみに絞りたい場合は、`isascii()` と組み合わせる方法もあります。

```
def validate_number(self, value_if_allowed):
    if value_if_allowed == "":
        return True
    return value_if_allowed.isascii() and value_if_allowed.isdigit()
```

8. validatecommandは1つのEntryにつき1つのみ

`bind()` とは異なり、`validatecommand` には `add="+"` のような複数登録の仕組みがありません。1つのEntryに設定できる検証関数は1つだけです。

```
# NG: 後から設定した方で上書きされてしまう
entry.config(validatecommand=vcmd_a)
entry.config(validatecommand=vcmd_b) # vcmd_a は消える
```

複数条件をまとめたい場合

1つの検証関数の中で、複数の条件をandでつなげてまとめます。

```
def _is_number(self, value):
    return value == "" or value.isdigit()

def _is_within_length(self, value, max_len=10):
    return len(value) <= max_len

def validate_number(self, value_if_allowed):
    return self._is_number(value_if_allowed) and self._is_within_length(value_if_allowed)
```

```
vcmd = (self.register(self.validate_number), "%P")
```

Entryの種類ごとに異なる組み合わせで検証したい場合は、窓口となる関数(validate_xxxなど)をEntryごとに用意し、それぞれの中で部品関数を組み合わせる設計にできます。

```
def validate_quantity(self, value):  
    return self._is_number(value) and self._is_within_length(value, max_len=5)  
  
def validate_price(self, value):  
    return self._is_number(value) and self._is_within_length(value, max_len=10)  
  
vcmd_quantity = (self.register(self.validate_quantity), "%P")  
vcmd_price = (self.register(self.validate_price), "%P")
```

まとめ

項目	ポイント
register()	Python関数をTcl/Tkが呼べる形式に変換。1回登録すれば使い回し可能
validate="key"	1文字入力・削除のたびに検証。リアルタイム制限向き
validate="focusout"	フォーカスが外れた時のみ検証。まとめてチェックしたい場合向き
%P	変更後のEntry全体の文字列を検証関数に渡す、最もよく使う記号
戻り値	Trueで入力許可、Falseでその入力(1文字)だけ拒否
複数条件	1つの関数内でandにまとめる。validatecommand自体は1つしか設定できない